

AI for Economic Research: Dynamic Models, Language, and Agents

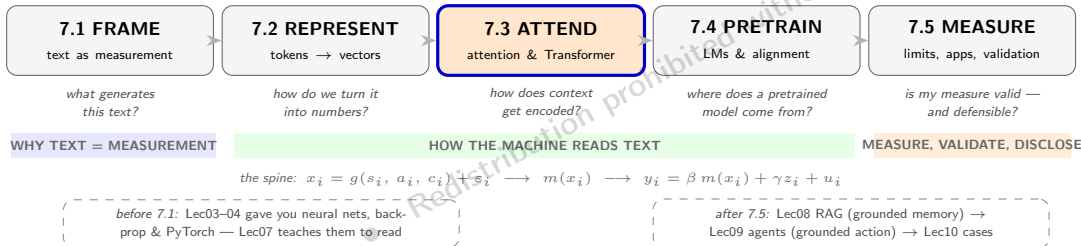
Lecture 7.3a: The Attention Mechanism

● Zhigang Feng

2026-07-07 05:00:08

Lecture 7 in One Map

you are here ▼



Route: one attention head, end to end (part 1/2 of ATTEND) — Q/K/V projections → scaled scores → softmax → weighted blend → multi-head; part 2/2 wraps the head into the full Transformer layer.

Why Context Needs a Mechanism

From “one vector per word” to “one vector per occurrence”

How the Transformer Works in Plain Language

- Consider the sentence: *“The Fed raised rates to combat inflation.”*
- When you read the word **rates**, you instinctively ask: *“rates of what? raised by whom? why?”* — your eye jumps back to *Fed* (subject) and forward to *inflation* (motivation).
- **A Transformer does exactly this, for every token simultaneously:**
 1. Every token asks: *“which other tokens are most relevant to understanding me?”*
 2. Each token collects a weighted blend of information from every other token in the sequence
 3. All positions are processed in **parallel** — not one-by-one as in an RNN
- Repeat this “everyone looks at everyone” operation L times (e.g. $L = 96$ in GPT-3) with different learned weights, and the model builds deeply contextualized representations.
- **Engineering name for this operation:** scaled dot-product self-attention. The next slides unpack the five steps in detail.
- **Contrast with Word2Vec / GloVe.** Those models produced *one vector per word type*, independent of context — *bank* in “river bank” and *bank* in “central bank” share the same **u**. A Transformer produces *one vector per token occurrence*, computed from its neighbors — the two *banks* end up at different points in $\mathbb{R}^d$. **Static** $\rightarrow$ **contextual** is the single conceptual jump.

The Transformer as a Function

- A Transformer is a function

$$\mathcal{T}: (\mathbb{R}^d)^n \longrightarrow (\mathbb{R}^d)^n, \quad [e^{(1)}, \dots, e^{(n)}] \mapsto [e'^{(1)}, \dots, e'^{(n)}]$$

mapping n input vectors to n output vectors of the same dimension. Inputs are token embeddings plus positional encodings (details in T3b); outputs are *contextualized* embeddings.

- **What “enriched” means.** Each $e'^{(i)} \in \mathbb{R}^d$ encodes at once (a) the identity of token $t^{(i)}$, (b) its grammatical/semantic role in *this* sequence, (c) a weighted blend of every other token's contribution (causally masked in autoregressive models).
- **Important:** these are the Transformer's direct output — an *intermediate representation*, **not the final answer**. Inside: L stacked layers, each containing self-attention and a feed-forward network.
- **Turning embeddings into task output.** A small *task head* sits on top:
 - Generation: project $e'^{(n)}$ onto the vocabulary $\rightarrow$ softmax $\rightarrow$ sample a token (Act 4).
 - Classification: project $e'^{([\text{CLS}])}$ onto class labels $\rightarrow$ softmax $\rightarrow$ pick a class.

Self-Attention, Step by Step

Five steps, one worked FOMC example

Self-Attention: The Key Idea

- For each position i in the sequence, ask: “*which other positions are most relevant to position i ?*”
- Compute a set of weights α_{ij} (with $\sum_j \alpha_{ij} = 1$) and form a new representation as a *weighted sum* of all positions.
- **Crucial properties:**
 - Every token can attend to every other token in one step (no recurrence)
 - Weights are computed dynamically from the input, not fixed
 - Operation is parallelizable across positions $\Rightarrow$ trains efficiently on GPUs
- This single operation replaces a tower of recurrent steps and is the central piece that made the Transformer work.

Step 1 — Project to Q, K, V

- For each input embedding $e^{(i)} \in \mathbb{R}^d$, compute three projections:

$$Q_i = W_Q e^{(i)} \quad (\text{query})$$

$$K_i = W_K e^{(i)} \quad (\text{key})$$

$$V_i = W_V e^{(i)} \quad (\text{value})$$

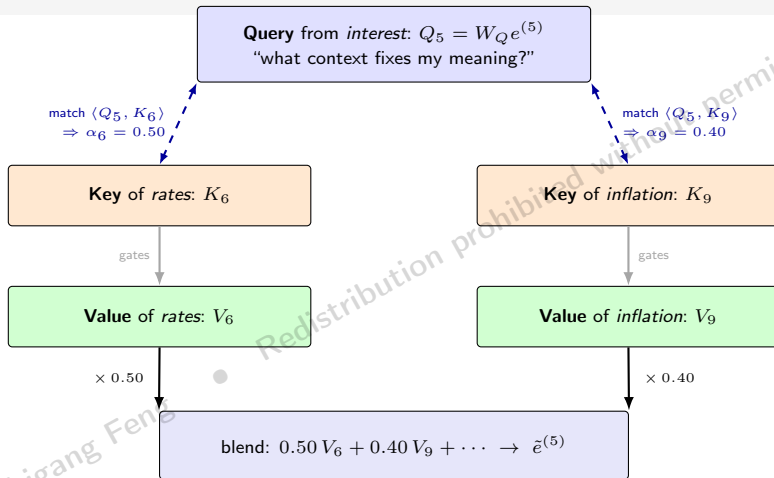
where $W_Q, W_K, W_V \in \mathbb{R}^{k \times d}$ are learned matrices and typically $k \leq d$.

- Mnemonic** (information retrieval):

- Q_i is the *question* that token i asks: “what context do I need?”
- K_j is the *description* that token j advertises: “here’s what I am about”
- V_j is the *actual content* that j contributes if matched

- What W_Q, W_K, W_V are trained to achieve:** shape the projections so that the dot product $\langle Q_i, K_j \rangle$ is high when V_j ’s content is useful to what the task needs at position i , and low otherwise. The model discovers which pairings matter from the loss; it is never told in advance.
- Why three separate projections?** Separating roles enables *asymmetric* attention: *rates* can ask a question (Q) that matches *inflation*’s advertisement (K) without *inflation* having to ask the same question. The next slide shows the three roles in action.

Q/K/V at a Glance — Match Sets the Weight, the Weight Reads the Value



Three roles, kept separate. The *query*–*key* match (dashed) only decides *how much* to read; what gets read is the *value*. Asymmetry matters: *interest* queries *rates* without *rates* querying back. W_Q, W_K, W_V are all learned from the loss — nobody hand-codes “attend to rates.”

The Retrieval Analogy: A Soft Database Lookup

Attention is a search engine that returns a weighted blend instead of one hit.

Retrieval system	Self-attention	At position i
search query	query vector	$Q_i = W_Q e^{(i)}$: “what do I need?”
document tags / index	key vectors	$K_j = W_K e^{(j)}$: “what I am about”
document contents	value vectors	$V_j = W_V e^{(j)}$: what is actually read
relevance ranking	scaled dot products	$\langle Q_i, K_j \rangle / \sqrt{k}$
click the top result	softmax blend	read <i>all</i> results, weighted

- Every position plays all three roles at once: it queries the others, advertises itself, and delivers content when matched.
- The “soft” part — a convex combination rather than a hard top-1 pick — keeps the whole lookup differentiable, so it can be *learned* end-to-end by gradient descent.
- Preview: the same query–key logic powers document retrieval in RAG (Lec08) — there the “keys” are your corpus.

Step 2 — Scaled Dot-Product Scores

- For each pair (i, j) , measure the compatibility between query Q_i and key K_j by their dot product, scaled:

$$s_j^{(i)} = \frac{\langle Q_i, K_j \rangle}{\sqrt{k}}.$$

- Why divide by $\sqrt{k}$?** As the dimension k grows, dot products tend to grow too, pushing softmax into saturation (one term dominates, gradients vanish). Scaling by $\sqrt{k}$ keeps the variance of $s_j^{(i)}$ around 1.
- In autoregressive models, scores to future positions ($j > i$) are set to $-\infty$ before the softmax — the **causal mask** (more in Step 3).
- These raw scores are the “relevance ranking” of the retrieval analogy — one number per candidate position, before normalization.

Step 3 — Softmax → Attention Weights

- Convert scores into a probability distribution over positions:

$$\alpha_j^{(i)} = \frac{\exp(s_j^{(i)})}{\sum_{r=1}^n \exp(s_r^{(i)})}.$$

- Properties:

- $\alpha_j^{(i)} \geq 0$ and $\sum_j \alpha_j^{(i)} = 1$
- Concentrated weights $\Rightarrow$ “hard” attention to a few positions
- Diffuse weights $\Rightarrow$ “soft” attention spread across many

- **Causal masking** (autoregressive models): when predicting position $i + 1$, set $s_j^{(i)} = -\infty$ for $j > i$ before the softmax. This prevents the model from peeking at the future during training.
- **Encoder models** (BERT) use bidirectional attention: every token sees every other.
- **Why softmax?** We want the output to be a *convex combination* of value vectors — weights that sum to 1, all non-negative. Softmax converts any real-valued score vector into exactly such a probability distribution, enabling an interpretable weighted average of content across the sequence.

Step 4 — Weighted Sum of Values

- Given the attention weights $\alpha_j^{(i)}$, the attention output for position i is the weighted sum of all value vectors:

$$\text{AttnVec}_i = \sum_{j=1}^n \alpha_j^{(i)} V_j \in \mathbb{R}^k.$$

- Interpretation:** token i “reads” content from all positions in the sequence, but pays most attention to those whose keys best match its query.
- Long-range, short-range, and same-position interactions all happen in the same step — contrast with RNNs where information has to propagate through many time steps.
- Note: V_j is what is being passed around. K_j only decides *how much* to pass; the content delivered is V_j .
- Why a weighted sum rather than a hard lookup?** Instead of selecting a single “most similar” token, the model retrieves a *blend*: for *interest* in a monetary-policy sentence, it mixes V_{rates} , $V_{\text{inflation}}$, and V_{raised} into one vector that reflects all three contexts simultaneously — richer than any single lookup could provide.

Step 5 — Project Back to Dimension d

- The attention output lives in $\mathbb{R}^k$, but we need a vector in $\mathbb{R}^d$ to feed the next layer. Project back with a learned matrix $W_O \in \mathbb{R}^{d \times k}$:

$$\tilde{e}^{(i)} = W_O \text{AttnVec}_i.$$

- This gives the contextualized representation of token i *after* self-attention.
- Why project back to $\mathbb{R}^d$?** In multi-head attention each head operates in a smaller subspace $\mathbb{R}^k$ (with $k = d/H$) to keep compute manageable. Projecting back with W_O after concatenating heads restores the common dimension d , so the result flows into the next layer without any shape change.
- Five-step recap:**
 - Project: Q_i, K_i, V_i
 - Score: $s_j^{(i)} = \langle Q_i, K_j \rangle / \sqrt{k}$
 - Softmax: $\alpha_j^{(i)}$
 - Weighted sum: $\text{AttnVec}_i = \sum_j \alpha_j^{(i)} V_j$
 - Output projection: $\tilde{e}^{(i)} = W_O \text{AttnVec}_i$
- All matrices (W_Q, W_K, W_V, W_O) are learned by backpropagation from a task loss.

Worked Example — Part 1

Sentence: “The central bank raised interest rates amid rising inflation.”

Tokenization (9 word tokens, numbered left to right):

The central bank raised interest rates amid rising inflation
1 2 3 4 5 6 7 8 9

Goal: produce $e^{(5)}$, the contextualized vector for *interest* — one that encodes how the word is used *in this sentence* (the interest of monetary-policy rates, not “curiosity” or “financial stake”).

- **Attention score** $s_j^{(5)} = \langle Q_5, K_j \rangle / \sqrt{k}$ measures how much *interest* should listen to token j . Suppose, after scaling, the model finds:

Token j	$s_j^{(5)}$
rates ($j = 6$)	2.3
inflation ($j = 9$)	1.9
raised ($j = 4$)	1.1
bank ($j = 3$)	0.5
the ($j = 1$)	-0.2

Intuitively: *interest* is most strongly tied to *rates* and *inflation* — the words that fix its meaning here.

Worked Example — Part 2

- **Softmax** $\rightarrow$ **attention weights**. $\alpha_j^{(5)}$ is the *fraction* of attention *interest* pays to token j — a probability, with $\sum_j \alpha_j^{(5)} = 1$:

$$\alpha_6^{(5)} \approx 0.50, \quad \alpha_9^{(5)} \approx 0.40, \quad \alpha_4^{(5)} \approx 0.05, \quad \dots$$

50 % of the attention goes to *rates*, 40 % to *inflation*, a sliver to everything else.

- **Weighted sum of value vectors** (what the token actually “reads”):

$$\text{AttnVec}_5 \approx 0.50 V_6 + 0.40 V_9 + 0.05 V_4 + \dots$$

- **Project back**: $\tilde{e}^{(5)} = W_O \text{AttnVec}_5$.
- **Result**: $\tilde{e}^{(5)}$ is a vector in $\mathbb{R}^d$ that represents “*interest, in a context dominated by rates and inflation*” — exactly the meaning we want for monetary-policy text.
- Swap the sentence to “The discount rates applied to pension funds were revised” and the same machinery shifts its reading: *discount* and *pension* become the high-weight sources, and the vector lands in discounting/pensions territory instead.
- The same computation runs in parallel for every position i , producing $\tilde{e}^{(1)}, \dots, \tilde{e}^{(9)}$ — every

How Q , K , V Are Learned

- **What is trainable (the “controls” θ):** in every layer, W_Q, W_K, W_V, W_O , plus the FFN weights W_1, W_2, b_1, b_2 , LayerNorm parameters, the token embedding matrix $\mathcal{E}$, and the output head W_{vocab} . All are initialized randomly and updated by gradient descent on a **single global loss**.
- **Notation:** $t^{(i)}$ is the i -th token of a training sequence s . The sum $\sum_s \sum_i$ runs over every position i in every sequence s in the corpus.

- **Common training objectives:**

- **Autoregressive LM** (GPT-style):

$$\mathcal{L}(\theta) = - \sum_s \sum_i \log P_{\theta}(t_s^{(i+1)} \mid t_s^{(1:i)}).$$

- **Masked LM** (BERT-style): hide tokens at random and predict them from bidirectional context.
- **Classification fine-tuning:** cross-entropy on a sentence-level label.
- **One loss, all parameters.** Backpropagation sends gradients through softmax, dot products, Q/K/V projections, residuals, LayerNorm, *and the FFN* — one backward pass updates every parameter together. There is no separate training signal for attention vs. the feed-forward block.
- Nothing tells the model “attend to neighbors” or “ignore stopwords” — it discovers attention patterns from data.

Multi-Head Attention

- One (Q, K, V) trio captures only one “view” of the sequence. **Multi-head attention** runs H such trios in parallel and concatenates their outputs.
- Each head h has its own projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, all of dimension $k = d/H$.
- After running H attention computations independently, concatenate and project:

$$\text{MultiHead}_i = W_O^* \left[\text{AttnVec}_i^{(1)}; \dots; \text{AttnVec}_i^{(H)} \right].$$

- **Do heads have different objectives?** *No — all H heads share the same training loss* (e.g. next-token cross-entropy). What differs is their random initialization. Specialization is **emergent**: under one global loss, H parallel feature detectors diversify because identical copies of the same head would be redundant.
- **Observed specializations in financial text:**
 - Head 1: monetary-policy relations (*inflation* $\leftrightarrow$ *interest*)
 - Head 2: temporal phrases (*Q1, next month, by year-end*)
 - Head 3: hawkish vs. dovish stance markers

Nobody told the heads what to look for — they were not assigned separate objectives.

- Total parameter cost is essentially the same as a single-head attention with the full dimension d

Thinkbox: Attention Entropy as a Measurement Diagnostic

The weight distribution itself carries information about measurement quality.

- Position i 's weights $\alpha^{(i)}$ form a probability distribution; its Shannon entropy $H(\alpha^{(i)}) = -\sum_j \alpha_j^{(i)} \log \alpha_j^{(i)}$ measures how *focused* the reading is.
- $H \approx 0$: meaning pinned down by a few strong context words — 2022-style tightening statements, where *inflation* attends to *raised*, *persistent*, *mandate*.
- $H \approx \log n$: attention spread thin — the contextual vector approaches a global average. 2019-style “crosscurrents” statements: *inflation* attends diffusely to *employment*, *growth*, *uncertainty*, ...

High-entropy passages produce noisier stance scores — measurement-error variance that *moves with the state of the economy*. If you regress on Transformer-based stance measures, attention entropy is a candidate flag for heteroskedastic measurement error. A PBOC wrinkle to discuss: boilerplate political formulas share every passage with the substantive policy words — does that push the attention entropy of terms like “inflation” systematically higher in PBOC reports than in FOMC statements?

Bridge: One Head Is a Building Block

We can now compute a context-aware vector — once.

- One attention pass mixes information *across* positions. On its own it is still linear in the values, blind to word order, and single-pass.
- **T3b wraps the head into a layer:** positional encodings (order), residual connections + LayerNorm (trainability at depth), a feed-forward network (nonlinearity — and, it turns out, the model's stored knowledge) — then stacks the layer L times and chooses among three architectural variants, ending with a full research case on 10-K filings.

Arc: $T1 \text{ FRAME} \rightarrow T2 \text{ REPRESENT} \rightarrow \mathbf{T3 \text{ ATTEND}} \rightarrow T4 \text{ PRETRAIN} \rightarrow T5 \text{ MEASURE}$.